The Engineers IDE

EXPLORER

- Proyecto_Curso/
  - .venv/
  - src/
  - .gitignore
  - pyproject.toml
  - README.md

main.py

Fundamentos y Entornos de Desarrollo en Python

De scripts amateur a ingeniería de software

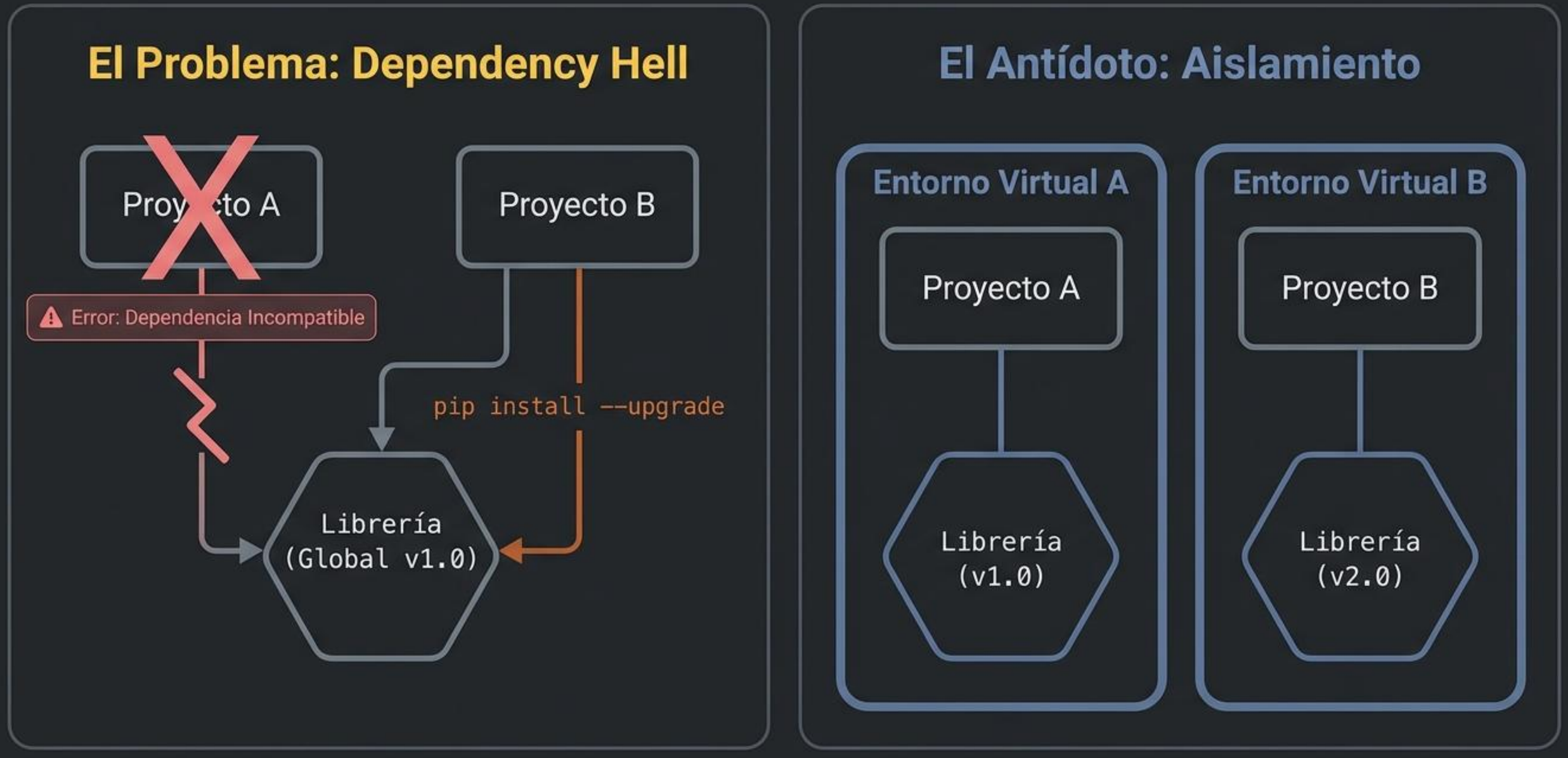
Docente: Msc. Jaime Antonio Huaytalla Pariona
Semana: 01

Terminal

```
user@system:~/Proyecto_Curso$ iniciar_entorno
```

Gemini Notebook

The Engineers IDE



Gemini Notebook

Aislamiento con Entornos Virtuales venv

venv es el estándar oficial del lenguaje desde Python 3.3 (formalizado en el PEP 405).

Crea un directorio autocontenido y aislado del Python del sistema operativo.

⚠ **Regla de Oro:** Nunca versionar. Añadir .venv al .gitignore inmediatamente.

La convención .venv garantiza la detección automática por los IDEs.



Gemini Notebook

Análisis Comparativo: venv vs. conda

	venv	conda
Instalación previa	Ninguna (incluido en Python)	Requiere Anaconda / Miniconda
Alcance	Solo paquetes Python (vía pip)	Paquetes Python y binarios no-Python (C, CUDA)
Peso / Tamaño	Ligero	Más pesado
Caso de uso típico	Aplicaciones web, scripts, APIs	Ciencia de datos, Machine Learning, dependencias complejas

Perspectiva de la Industria: Herramientas todo-en-uno ultrarrápidas como uv y Poetry están dominando el ecosistema moderno, pero venv sigue siendo la base pedagógica esencial para comprender el mecanismo subyacente de aislamiento.

Gemini Notebook

La Evolución de las Dependencias

El Pasado Fragmentado



requirements.txt
plano, sin metadatos

setup.py
ejecutable, riesgo de seguridad

setup.cfg
configuración esparcida

Consolidación
(PEP 518, 621, 631)

El Estándar Moderno



- Centraliza metadatos, dependencias y configuración de herramientas en un único archivo declarativo.

- Mejores Prácticas: Fija versiones (==) para reproducibilidad total en producción, o usa rangos (>=) para mantener flexibilidad en bibliotecas.

Gemini Notebook

Anatomía de pyproject.toml

```toml
[build-system]
requires = ["setuptools>=61.0"]
build-backend = "setuptools.build_meta"

[project]
name = "mi-proyecto"
version = "0.1.0"
dependencies = [
    "requests==2.31.0",
    "pandas>=2.0.0"
]
```

Terminal

```
$ pip install -e .
```

Instalación editable. Permite importar el paquete en desarrollo y reflejar cambios en el código inmediatamente sin tener que reinstalar la librería.

Gemini Notebook

Control de Versiones: Git + Python

Fundamentos

Git registra fotografías (commits) incrementales del estado del código.

El Staging Area prepara y aísla los cambios específicos a incluir.

Regla de oro: No versionar basura, artefactos compilados, ni secretos.

.gitignore

```
# Entornos virtuales
.venv/
env/

# Artefactos compilados
__pycache__/
*.pyc

# Secretos y variables de entorno
.env
```

Consistente con la plantilla oficial de GitHub para Python.

Gemini Notebook

Estrategias de Ramificación (GitHub Flow)



Trunk-Based

(Integración múltiple diaria).

Ramas de vida muy corta. Exige equipos muy maduros.

Git Flow

(Ramas release / hotfix / develop).

Útil para software con versiones empaquetadas lentas.

GitHub Flow

(El modelo ilustrado).

Recomendado para entregas continuas. Ramas cortas + Pull Requests.

Gemini Notebook

Automatización y Trazabilidad: Commits ↔ SemVer

La especificación Conventional Commits (`<tipo>[ámbito]: <descripción>`) permite generar changelogs y calcular incrementos de versión de forma automática.

`fix: resolve api timeout` → Incrementa `PATCH` (`v1.0.0` ➔ `v1.0.1`)
Correcciones de errores que no rompen la compatibilidad.

`feat: add user authentication` → Incrementa `MINOR` (`v1.0.1` ➔ `v1.1.0`)
Nuevas funcionalidades compatibles con la versión anterior.

`feat!: change database schema` ○ BREAKING CHANGE → Incrementa `MAJOR` (`v1.1.0` ➔ `v2.0.0`)
Cambios drásticos que rompen la API y exigen intervención.

Gemini Notebook

Higiene y Seguridad del Proyecto

Los Tres Pilares

1. Commits Atómicos: Un cambio lógico por commit. Facilita aislar errores con `git bisect` o revertir archivos específicos.

2. Protección de Ramas: Nunca hacer `force-push` sobre `main`. Forzar la validación de código exclusivamente mediante Pull Requests.

3. Cero Secretos en Código: Jamás versionar contraseñas, API keys o tokens. Usar variables de entorno inyectadas.



Gemini Notebook

Arquitectura de Código: Flat vs. Src Layout

Flat Layout

(Para scripts rápidos o inicio de curso)



- Pros: Simple, rápido de inicializar.
- Contras: Python puede importar accidentalmente la copia local en lugar de la versión instalada en el entorno, ocultando errores de dependencias.

Src Layout

(El estándar profesional PyPA / pyOpenSci)



- Pros: Obliga a realizar una instalación editable para ejecutar el código, garantizando que el entorno de desarrollo se comporte idénticamente al de producción.

Gemini Notebook

Legibilidad y Estilo: PEP 8

Indentación y Longitud

4 espacios exactos (jamás mezclar tabs).
Límite orientativo de 79 caracteres por línea.

Nomenclatura (Convenciones)

`snake_case` (variables/funciones)
`PascalCase` (clases)
`UPPER_CASE` (constantes)

Docstrings

Resumen de una línea, línea en blanco, seguido del detalle extenso.

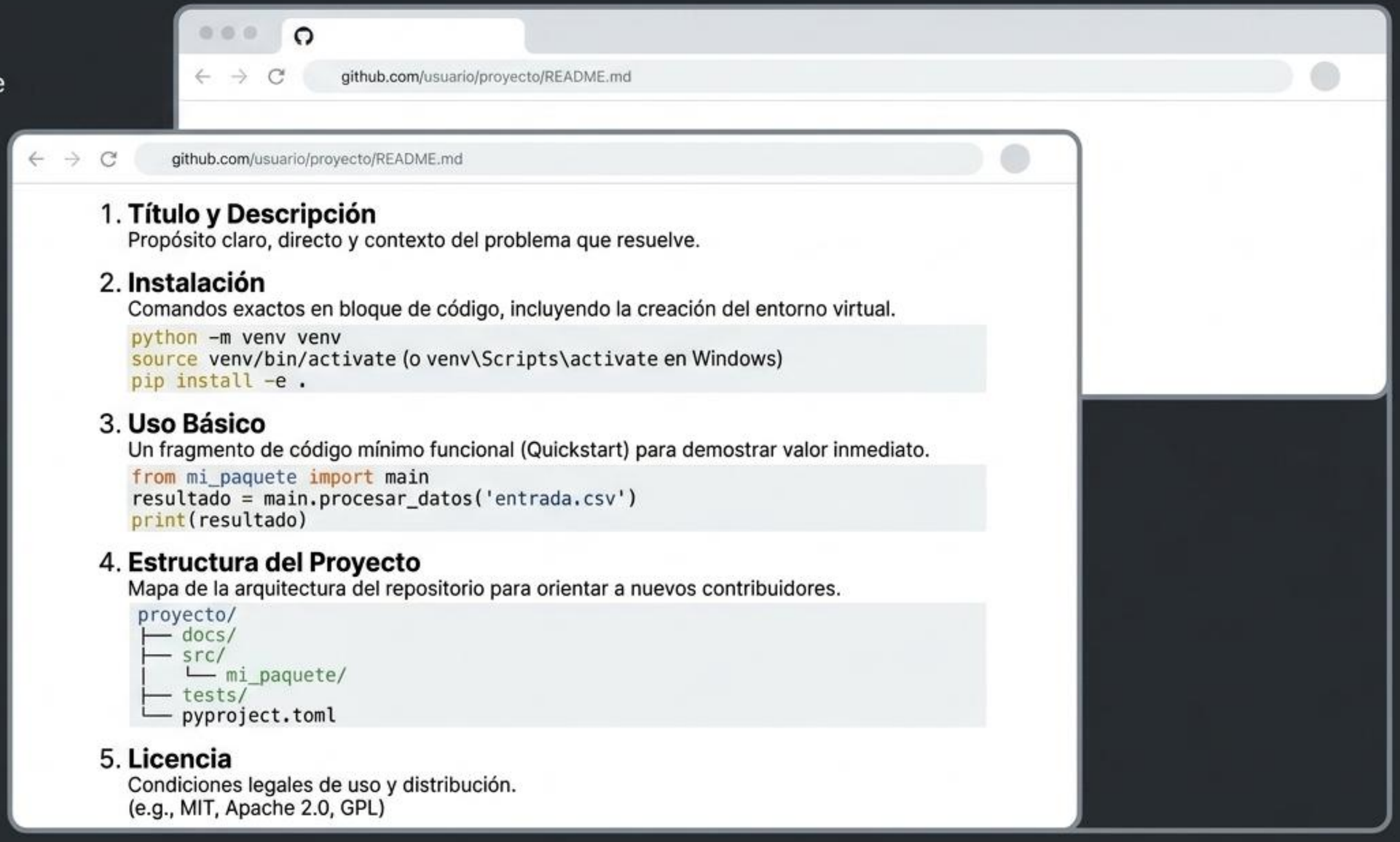
> "La consistencia dentro de un proyecto es más importante que la letra estricta de la norma." - Guido van Rossum

Automatización: No lo hagas a mano. Utiliza verificadores (pycodestyle) o formateadores automáticos inflexibles (Black, autopep8).

Gemini Notebook

El Punto de Entrada: README.md

Regla de los 2 minutos:
Un buen README debe responder inmediatamente **¿Qué hace?, ¿Cómo lo instalo?** y **¿Cómo lo uso?**



Gemini Notebook

Actividad Práctica: Construyendo la Anatomía

Aplica estas especificaciones según las indicaciones del docente

Mission_Checklist.sh

```
[ ] 1. Inicializar repositorio Git y añadir .gitignore oficial para
       Python.
[ ] 2. Crear entorno virtual (.venv) y activarlo en la terminal.
[ ] 3. Instalar librería de prueba (requests) y generar
       pyproject.toml mínimo.
[ ] 4. Estructurar en Flat Layout e instalar el proyecto en modo
       editable (pip install -e .).
[ ] 5. Redactar README.md con las 5 secciones clave.
[ ] 6. Crear una rama feature, realizar 3 commits con Conventional
       Commits y fusionar a main.
```

Gemini Notebook

Referencias y Bibliografía

Python Software Foundation. (2023). PEP 405 – Python Virtual Environments.

PEP 8 – Style Guide for Python Code. (peps.python.org).

Conventional Commits Specification v1.0.0. (conventionalcommits.org).

Python Packaging Authority (PyPA). src layout vs. flat layout.

Anaconda, Inc. (2024). Managing environments.

daily.dev (2026). Git best practices: what actually matters on real teams.

Lutz, M. (2013). Learning Python (5.ª ed.). O'Reilly.

Percival, H. & Gregory, B. (2020). Test-Driven Development with Python (2.ª ed.). O'Reilly.

Gemini Notebook